

```

1 <#
2   Redact-FileContent.ps1
3
4   Unified replacement/redaction engine for files, recursively through subfolders.
5   All modes below share the same file-processing, dry-run and backup logic.
6
7   1. Literal/regex replacements ($Replacements below) - always applied.
8       Each key is a regex pattern, each value is what to replace it with.
9       (Companion to Search-FilesForWord.ps1 - use that one first to preview matches.)
10
11   2. -AnonymizeSwitchNames - finds whitespace-delimited tokens that start with
12       "S"/"s" and end with "nn" (letters, digits, and special characters allowed
13       in between, e.g. S-DC1-01nn) and replaces each with a sequential generic
14       placeholder (SWITCH001, SWITCH002, ...).
15
16   3. -AnonymizeUsernames - finds tokens like "user22351" (literal "user" followed
17       directly by digits) and replaces each with a sequential placeholder
18       (USER001, USER002, ...).
19
20   4. -AnonymizeIPs - finds IPv4 addresses and replaces each with a sequential
21       placeholder (IP001, IP002, ...).
22
23   5. -StripPortDescriptions - removes the description text from interface lines
24       such as "GigabitEthernet1/0/1 - TO_SWITCH001", leaving just the port name
25       ("GigabitEthernet1/0/1"). Covers GigabitEthernet, TenGigabitEthernet,
26       TwentyFiveGigabitEthernet, FastEthernet and Ethernet ports.
27
28   For modes 2-4, the same original value always maps to the same placeholder
29   within a single run (each category has its own counter), but nothing is
30   persisted between runs - running the script again assigns fresh numbers.
31
32   All write modes support -DryRun (report what would change, touch nothing) and
33   -Backup (copy each modified file's original content aside before overwriting it).
34
35   USAGE EXAMPLES:
36
37       # Preview the literal replacements only, no files modified
38       .\Redact-FileContent.ps1 -Path "D:\" -DryRun
39
40       # Apply literal replacements for real, keeping backups
41       .\Redact-FileContent.ps1 -Path "D:\" -Backup
42
43       # Anonymize switch names, usernames and IPs, and strip port descriptions
44       .\Redact-FileContent.ps1 -Path "D:\" -AnonymizeSwitchNames -AnonymizeUsernames -AnonymizeIPs -StripPortDescriptions -Backup
45
46       # Anonymization only - skip the literal $Replacements table
47       .\Redact-FileContent.ps1 -Path "D:\" -AnonymizeIPs -Replacements @{} -DryRun
48
49       # Limit to specific file types
50       .\Redact-FileContent.ps1 -Path "D:\Logs" -Include *.log,*.txt -AnonymizeUsernames -DryRun
51 #>
52
53 [CmdletBinding(SupportsShouldProcess = $true)]
54 param(
55     # Root folder to process. Defaults to the current directory.
56     [string]$Path = ".",
57
58     # Optional file name filters (wildcards), e.g. *.txt, *.log, *.csv.
59     # Leave empty to process every file.
60     [string[]]$Include = @("*"),
61
62     # Ordered map of regex pattern -> replacement text, applied on every line.
63     [System.Collections.Specialized.OrderedDictionary]$Replacements = [ordered]@{
64         "ass" = "aaa"
65     },
66

```

```

67 # Anonymize switch/device name tokens matching -SwitchNamePattern.
68 [switch]$AnonymizeSwitchNames,
69 [string]$SwitchNamePattern = '(?!\\S)[Ss][^\\s]*nn(?!\\S)',
70 [string]$SwitchNamePrefix = "SWITCH",
71
72 # Anonymize username tokens matching -UsernamePattern.
73 [switch]$AnonymizeUsernames,
74 [string]$UsernamePattern = '(?!\\S)user\\d+(?!\\S)',
75 [string]$UsernamePrefix = "USER",
76
77 # Anonymize IPv4 addresses.
78 [switch]$AnonymizeIPs,
79 [string]$IPPattern = '(?!\\d)(?:(:25[0-5]|2[0-4]\\d|1?\\d?\\d)\\.){3}(?:25[0-5]|2[0-4]\\d|1?\\d?\\d)(?!\\d)'
80 [string]$IPPrefix = "IP",
81
82 # Strip the description text off interface lines, e.g.
83 # "GigabitEthernet1/0/1 - TO_SWITCH001" -> "GigabitEthernet1/0/1".
84 [switch]$StripPortDescriptions,
85 [string]$PortDescriptionPattern = '^(<port>(?:TwentyFiveGigabitEthernet|TenGigabitEthernet|GigabitEthernet|FastEthernet|Ethernet)\\d+(?:/\\d+)*\\s*~\\s*\\.\\$',
86
87 # Report matches and what would change, without modifying any file.
88 [switch]$DryRun,
89
90 # Copy each file's original content into -BackupFolder before overwriting it.
91 [switch]$Backup,
92
93 # Folder to hold backups when -Backup is used. Defaults to a timestamped
94 # folder created next to -Path. Relative structure under -Path is preserved.
95 [string]$BackupFolder
96 )
97
98 if (-not (Test-Path -LiteralPath $Path)) {
99     Write-Error "Path not found: $Path"
100     return
101 }
102
103 if ($Backup -and -not $BackupFolder) {
104     $BackupFolder = Join-Path (Resolve-Path -LiteralPath $Path) "_backup_$(Get-Date -Format 'yyyyMMdd_HHm
mss')"
105 }
106
107 $resolvedRoot = (Resolve-Path -LiteralPath $Path).ProviderPath
108 $allFiles = Get-ChildItem -Path $Path -Include $Include -Recurse -File -ErrorAction SilentlyContinue
109
110 $ignoreCase = [System.Text.RegularExpressions.RegexOptions]::IgnoreCase
111 $switchNameRegex = [regex]::new($SwitchNamePattern, $ignoreCase)
112 $usernameRegex = [regex]::new($UsernamePattern, $ignoreCase)
113 $ipRegex = [regex]::new($IPPattern)
114 $portDescriptionRegex = [regex]::new($PortDescriptionPattern, $ignoreCase)
115
116 # Original value (as first encountered) -> generic placeholder, one map per category.
117 # Consistent for this run only - not persisted between runs.
118 $switchNameMap = [ordered]@{}
119 $usernameMap = [ordered]@{}
120 $ipMap = [ordered]@{}
121 $switchNameIndex = 1
122 $usernameIndex = 1
123 $ipIndex = 1
124
125 function Get-PlaceholderName {
126     param(
127         [string]$OriginalValue,
128         [System.Collections.Specialized.OrderedDictionary]$Map,
129         [string]$Prefix,
130         [string]$IndexVarName

```

```
131 )
132
133 if (-not $Map.Contains($OriginalValue)) {
134     $index = Get-Variable -Name $IndexVarName -Scope Script -ValueOnly
135     $Map[$OriginalValue] = "{0}{1:D3}" -f $Prefix, $index
136     Set-Variable -Name $IndexVarName -Scope Script -Value ($index + 1)
137 }
138 return $Map[$OriginalValue]
139 }
140
141 $changedCount = 0
142 $scannedCount = 0
143
144 foreach ($fileItem in $allFiles) {
145     $scannedCount++
146
147     $content = Get-Content -LiteralPath $fileItem.FullName -ErrorAction SilentlyContinue
148     if ($null -eq $content) {
149         continue
150     }
151
152     $fileMatchCount = 0
153     $updatedContent = foreach ($row in $content) {
154         $currentRow = $row
155
156         foreach ($search in $Replacements.Keys) {
157             $lineMatches = [regex]::Matches($currentRow, $search).Count
158             if ($lineMatches -gt 0) {
159                 $fileMatchCount += $lineMatches
160                 $currentRow = $currentRow -replace $search, $Replacements[$search]
161             }
162         }
163
164         if ($StripPortDescriptions -and $portDescriptionRegex.IsMatch($currentRow)) {
165             $fileMatchCount++
166             $currentRow = $portDescriptionRegex.Replace($currentRow, '${port}')
167         }
168
169         if ($AnonymizeSwitchNames) {
170             $currentRow = $switchNameRegex.Replace($currentRow, {
171                 param($match)
172                 $script:fileMatchCount++
173                 Get-PlaceholderName -OriginalValue $match.Value -Map $switchNameMap -Prefix $SwitchNamePr
174             })
175         }
176
177         if ($AnonymizeUsernames) {
178             $currentRow = $usernameRegex.Replace($currentRow, {
179                 param($match)
180                 $script:fileMatchCount++
181                 Get-PlaceholderName -OriginalValue $match.Value -Map $usernameMap -Prefix $UsernamePrefix
182             })
183         }
184
185         if ($AnonymizeIPs) {
186             $currentRow = $ipRegex.Replace($currentRow, {
187                 param($match)
188                 $script:fileMatchCount++
189                 Get-PlaceholderName -OriginalValue $match.Value -Map $ipMap -Prefix $IPPrefix -IndexVarNa
190             })
191         }
192
193         $currentRow
194     }
```

```
195
196     if ($fileMatchCount -eq 0) {
197         continue
198     }
199
200     $changedCount++
201
202     if ($DryRun) {
203         Write-Host "[DryRun] Would update: $($fileItem.FullName) ($fileMatchCount match(es))" -ForegroundColor
        Color DarkYellow
204         continue
205     }
206
207     if (-not $PSCmdlet.ShouldProcess($fileItem.FullName, "Replace $fileMatchCount match(es)")) {
208         continue
209     }
210
211     if ($Backup) {
212         $relativePath = $fileItem.FullName.Substring($resolvedRoot.Length).TrimStart('\', '/')
213         $backupPath = Join-Path $BackupFolder $relativePath
214         $backupDir = Split-Path -Path $backupPath -Parent
215         if (-not (Test-Path -LiteralPath $backupDir)) {
216             New-Item -ItemType Directory -Path $backupDir -Force | Out-Null
217         }
218         Copy-Item -LiteralPath $fileItem.FullName -Destination $backupPath -Force
219     }
220
221     Write-Host "Working on: $($fileItem.FullName) ($fileMatchCount match(es))" -ForegroundColor DarkYellow
222     $updatedContent | Set-Content -LiteralPath $fileItem.FullName
223 }
224
225 Write-Host ""
226 if ($DryRun) {
227     Write-Host "Dry run complete. $changedCount of $scannedCount file(s) would be updated." -ForegroundColor
        Color Green
228 } else {
229     Write-Host "Complete successfully! $changedCount of $scannedCount file(s) updated." -ForegroundColor
        Color Green
230     if ($Backup) {
231         Write-Host "Backups saved under: $BackupFolder" -ForegroundColor Green
232     }
233 }
234
235 function Write-NameMapping {
236     param([string]$Title, [System.Collections.Specialized.OrderedDictionary]$Map)
237
238     if ($Map.Count -eq 0) {
239         return
240     }
241     Write-Host "`n$Title mapping for this run:" -ForegroundColor Cyan
242     $Map.GetEnumerator() | ForEach-Object {
243         Write-Host ("  {0} -> {1}" -f $_.Key, $_.Value)
244     }
245 }
246
247 if ($AnonymizeSwitchNames) { Write-NameMapping -Title "Switch name" -Map $switchNameMap }
248 if ($AnonymizeUsernames) { Write-NameMapping -Title "Username" -Map $usernameMap }
249 if ($AnonymizeIPs) { Write-NameMapping -Title "IP address" -Map $ipMap }
```